RAGFailBench: Automated Construction and Controllable Failure-Injection Benchmarking for Retrieval-Augmented Generation

Taixing Bi
Independent Researcher
New York, NY, USA
taixingbi@gmail.com

Abstract—Most RAG benchmarks are released once and go stale as the underlying knowledge base changes. RAGFailBench instead regenerates a failure-aware benchmark whenever the corpus changes. It builds provenance-preserving question-answering (QA) seeds from a document corpus, validates them with rule checks, an LLM judge and a baseline-model test, audits a sample by hand, and injects six parameterized failure operators under a fixed eight-passage context budget. Each answer-expected failure case is compared with its own parent clean seed, so a drop is attributable to one operator. On a 768-page Wikipedia pilot (12,366 chunks), three seeded runs give a stable construction funnel (QA acceptance $74.8\% \pm 1.0\%$; failure verification $94.9\% \pm 0.3\%$). In a human audit of a 50-seed sample with automatic verdicts hidden, the primary annotator accepts 84.0% of seeds (95% CI 71.5–91.7%) and judges 96.3% of 54 sampled failure cases valid. Evaluating three Bedrock-hosted models shows that conflicting evidence is the dominant failure for all of them (26.6–30.4-point paired drop), that they abstain correctly on 92–98% of evidence-absent cases, and that context noise and evidence position cause no measurable drop with eight passages. On two Wikipedia snapshots three months apart, the regression gate passes the corpus update and blocks a retrieval-depth change (top-k 8→4) that leaves clean accuracy unchanged. Code, configurations and all run artifacts are released.

Index Terms—retrieval-augmented generation, benchmark construction, regression testing, failure injection, LLM evaluation, reproducibility

I. Introduction

Knowledge bases behind retrieval-augmented generation (RAG) systems change continuously: pages are edited, reorganized and retired, and every change is followed by re-indexing and redeployment. In practice such deployments are often checked against a small, manually written "golden" QA set that is rarely revisited. That set says little about the failure modes a RAG system meets in production: evidence that has been deleted, passages that look relevant but do not contain the answer, answers split across chunk boundaries, and documents that contradict each other after a partial update.

Existing RAG benchmarks do not close this gap. The modeling paradigm is well established [1]–[4], but benchmarks and evaluation frameworks such as RAGAS, ARES, RGB, RAGBench and CRUD-RAG score systems against data fixed at release time [6]–[10]. Tools that generate test sets from a user corpus, such as the RAGAS test-set generator and YourBench [6], [11], produce clean questions but no typed, controllable failure conditions. Teams therefore cannot answer a recurring question: after this week's document changes, did robustness regress, and against which failure mode?

RAGFailBench addresses this with (i) automatic construction of validated, provenance-preserving QA seeds from a corpus snapshot and (ii) six typed, parameterized failure operators that turn each seed into labeled failure cases. Models are evaluated on clean and failure conditions under the same context budget, and each answer-expected failure case is compared with its own parent seed.

A. Contributions

C1, automated construction. A configuration-driven pipeline that ingests a corpus, generates QA pairs with supporting-sentence provenance, filters them with documented rule, judge and baseline checks, and audits a stratified sample by hand.

C2, controlled failure injection. Six operators anchored to the RAG pipeline stage they corrupt, each with three severities, a fixed eight-passage budget, automatic leak checks and an independent LLM audit of every abstain-expected case.

C3, regression workflow, tested. The benchmark is regenerated from a new corpus snapshot and compared with a gate that reports a per-operator decision with bootstrap CIs. We run it on two Wikipedia snapshots and on a deliberately degraded retrieval configuration.

We ask RQ1: how much does each operator cost, measured against matched parents? RQ2: do models differ in sensitivity to specific operators? RQ3: does a regenerated benchmark catch a regression that clean accuracy misses?

II. Related Work

RAG and its evaluation. Lewis et al. introduced RAG [1]; in-context retrieval augmentation works without changing the model [5]; surveys chart its architectures [2]–[4]. RAGAS computes reference-free faithfulness and relevance scores [6], ARES trains lightweight judges on

TABLE I
Capability comparison with prior RAG evaluation systems.

System	Auto. constr.	Typed failures	Regenerable	Regr. gate
RAGAS [6]	partial	–	–	metrics only
ARES [7]	synthetic	–	–	metrics only
RGB [8]	partial	partial	–	–
RAGBench [9]	✓	–	–	–
CRUD-RAG [10]	✓	partial	–	–
YourBench [11]	✓	–	✓	–
RAGFailBench	✓	✓ (6)	✓	✓

synthetic data [7], RGB probes noise robustness, negative rejection and counterfactual robustness [8], and RAGBench and CRUD-RAG extend domain and update coverage [9], [10]. All evaluate against a fixed dataset. RAGFailBench treats the dataset itself as regenerable and every stress condition as a typed operator with a matched clean parent.

Stressors studied in isolation. Each operator reimplements, inside one regenerable pipeline, a stressor with its own literature. Position effects in long contexts were shown by Liu et al. [12] (evidence_position); the effect of irrelevant and related passages on RAG by Cuconasu et al. [13] (context_noise, hard_negative); and conflicts between retrieved evidence and parametric knowledge by Longpre et al. [14] and ClashEval [15] (conflict). Our contribution is not a new stressor but the construction, validation and regeneration around them.

QA construction and retrieval. SQuAD, Natural Questions, TriviaQA, HotpotQA and KILT established the formats most RAG evaluation still uses [16]–[20] but were built once by large annotation efforts. RAGFailBench automates supporting-sentence QA construction so it can be rerun on a new snapshot. Sparse and dense retrievers [21]–[25] and BEIR [26] evaluate retrieval itself; RAGFailBench does not run a retriever and does not measure Recall@k, MRR or nDCG (Section IV).

LLM judges and tooling. We use an LLM correctness judge in the spirit of G-Eval [27], and validate it against human labels. Industry tools such as LangSmith, OpenAI Evals, DeepEval and Promptfoo [28]–[31] execute test sets but assume the set already exists; holistic benchmarks such as HELM and MTEB [32], [33] motivate per-condition rather than single-score reporting. Our regeneration loop follows continuous-integration and regression-test-selection practice [34], [35].

III. Framework

RAGFailBench is a linear pipeline (Fig. 1): a corpus snapshot is filtered and chunked; candidate QA pairs are generated and validated automatically; a sample of the accepted seeds is audited by hand; seeds are turned into failure cases; and clean and failure cases are evaluated and compared with a previous run.

A. Corpus

The pilot corpus is fetched through the MediaWiki API [41]: 1,000 English Wikipedia pages [40] (200 per category: person, location, science/technology, historical event, organization/product). Disambiguation and stub pages are removed and pages are deduplicated, leaving 768 pages, which section-aware chunking splits into 12,366 chunks. Every page's revision id is recorded, so a snapshot can be re-fetched exactly. The source module is a pluggable adapter; only Wikipedia is evaluated here.

B. QA generation

Candidate QA pairs are generated per chunk by Qwen3-32B (Table III), which returns a question, an answer and the supporting sentence that justifies it. The prompt names the source article and requires the question to name its subject ("When did Albert Einstein die?", not "When did he die?"). Retaining the supporting sentence is what lets the absence operators remove exactly the justifying evidence. Generation follows a difficulty quota (40% easy, 40% medium, 20% hard).

C. Automatic validation

Candidates first pass deterministic rule checks on normalized text: uniqueness (a who/which/what question with a person or organization answer whose supporting sentence lists other entities joined by "and"), containment (the gold answer occurs in the supporting sentence, which occurs in the chunk), refers-to-passage ("the passage", "mentioned"), time-sensitive phrasing ("current", "latest", an undated "as of"), answer in question, unresolved referent (a pronoun with no name before it, or a possessive or "the company"-style phrase in a question that names nothing), title-only (a question that only asks what the page title is), multi-part ("when and where", "…, and what") and length limits. Rule-passing candidates are then assessed in parallel by an LLM answerability judge (answerable, supported, unique, clear, with a confidence) and a baseline test in which a reference model answers from the gold chunk alone. Accepted candidates are deduplicated (exact or character-5-gram Jaccard ≥ 0.85) and a stratified sample of 200 per run becomes the clean seeds.

The quality score $q = 0.3\,[\text{rules}] + 0.1\,[\text{unique}] + 0.3\,c\,(a{+}s{+}u{+}k)/4 + 0.3\max(F_1^{\text{base}}, [\text{base correct}])$ combines these signals (a, s, u, k are the judge's verdicts and c its confidence). A threshold sweep shows that q never makes a decision on its own: every candidate passing the hard gates scores ≥ 0.985, and a rule failure caps q at 0.70, so any threshold from 0 to 0.95 accepts the same candidates. We report q as a ranking score, not a filter. The multi-part rule was added after the runs and applied to the selected seeds, removing 11, 16 and 12 of 200 (with all their failure cases and evaluation rows); every drop is paired by parent, so removing a parent keeps comparisons matched.

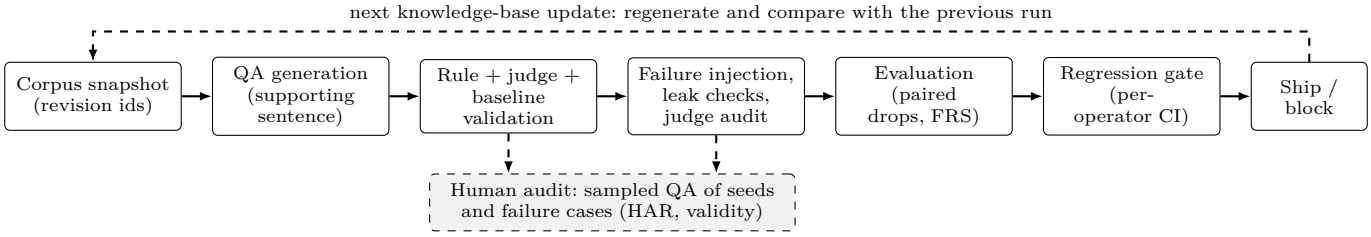


Fig. 1. The RAGFailBench loop. A knowledge-base update triggers re-ingestion, automatic regeneration of seeds and failure cases, evaluation, and a gated decision. The human audit (dashed) is sampled quality assurance of accepted seeds and failure cases; it reports HAR and failure validity and does not approve every item.

D. Human audit (sampled quality assurance)

Automatic validation is not sufficient on its own (Section V-B), so the workflow includes a human stage. It is sampled quality assurance: after each regeneration, a stratified sample of seeds and failure cases is exported with all automatic verdicts hidden, labeled by annotators against a written guide (keep / fix / reject with a reason for every fix or reject; per-operator validity and severity), and summarized as a human acceptance rate (HAR) and failure validity with confidence intervals. The audit measures the quality of a regenerated benchmark and feeds new automatic rules; it does not approve each item, and the pipeline does not wait for it to produce a benchmark. Teams that need item-level approval can label the full export with the same sheets.

E. Failure operators

Each operator keeps the question and gold answer, rewrites the context, declares an expected behavior and is instantiated at three severities (Table II). Every clean and failure case presents exactly eight passages: slots an operator does not use are filled with off-topic passages from other pages, checked not to contain any surface form of the answer. Median context length is 5.2k–8.1k characters for every condition (clean 6.3k).

For the abstain-expected operators, a case is quarantined if any passage contains the gold answer in any surface form (bracket, date-order, comma-head and type-suffix variants). An LLM judge independent of the generator then audits every remaining case and quarantines it if the answer can still be inferred; it rejects 12–20% of high- and 6–10% of medium-severity missing_evidence cases and 4–5% of hard_negative cases. Conflict alternates are, in order of preference, a small perturbation of a date or number (40% of cases), an alternate written by the generator and rejected by the judge if it is also correct (56%), or a same-type answer from another seed (4%).

Example (s42, seed_000008). “Which book contains lectures delivered by Richard Feynman for lay audiences in 1965?” (gold: The Character of Physical Law). All three models answer the clean case correctly and abstain when the evidence is removed. In the medium conflict case, whose copy names The Physics of Light and Matter, nova-pro answers the alternate, llama names both, and gpt-oss answers gold: three different conflict behaviors on one item.

IV. Experimental Setup

Scoring. Each item is scored against its expected behavior. Answer-expected items are correct if any of exact match, token-F1 ≥ 0.6 or the judge accepts the answer. The judge follows the G-Eval pattern of a rubric prompt to a strong LLM [27]: it sees the question, gold answer and prediction and returns strict JSON {correct, confidence}, with minor formatting differences allowed. The pipeline refuses to run if the judge is an evaluated model, and a failed or unparseable judge call is excluded and counted, never scored as wrong (0 in the final runs). On clean items the judge alone accepts 97–98% of answers, token-F1 84–88% and exact match 69–75%; the union adds about one point over the judge. Abstain-expected items (missing_evidence, hard_negative) are scored on correct abstention.

Conflict scoring. Each conflict answer gets one outcome. Abstain if the model abstains; otherwise the gold and alternate values are matched in the normalized answer (the alternate is masked first, so a gold “15” does not match inside “150”), and the answer is both if it contains both values, gold if it is correct by the scoring rule above or contains the gold value, alternate if it contains only the alternate, and other otherwise. An answer acknowledges the conflict if it is both or uses a contradiction cue (“conflict”, “contradict”, “inconsistent”, “discrepancy”, “differ”). The reported score is the handled rate: gold, both, or an abstain/other answer that acknowledges the conflict. This rule is deliberately lenient: it credits surfacing the contradiction, the behavior a RAG system with a stale document should show, and it avoids penalizing two-value answers that a single-answer judge marks wrong. Because the rule changes the numbers, we also report two stricter rules, gold only and answer correctness without outcome credit (Table VI).

Paired drops and FRS. For an answer-expected item, the drop is its parent clean seed’s correctness minus its own (each 0 or 1), so a drop is not confounded by which seeds an operator could be built from. For an abstain-expected item, whose parent expects an answer, the drop

TABLE II
The six failure operators. Every case has eight passages; unused slots hold off-topic padding.

Operator	Stage	Expected	Mechanism and severity (low / medium / high)	Literature
missing_evidence	retrieval / evidence	abstain	All evidence removed; remaining context is other pages / up to two neighbouring passages of the source page (none in 11% of cases) / the gold chunk with the answer sentence deleted	RGB [8]
hard_negative	retrieval	abstain	Gold chunk replaced by 2 / 4 / 8 topically similar passages without the answer	[13]
conflict	knowledge state	answer or flag	Gold chunk plus a copy identical except for the answer; gold in slot 1, copy in slot 2 / copy in slot 1, gold in slot 2 / copy in slots 1 and 7, gold in slot 8	[14], [15]
context_noise	context assembly	answer	Gold plus seven others, of which 2 / 4 / 6 are topically similar distractors	[13]
chunk_boundary	chunking	answer	Supporting sentence split into 2 adjacent pieces / 2 pieces with a distractor between / 3 pieces with distractors between	–
evidence_position	context assembly	answer	Gold passage in slot 1 / the middle / slot 8	[12]

TABLE III
Experimental configuration.

Item	Setting
Corpus	768 pages, 12,366 chunks (snapshot A, 2026-08-01)
Seeds	1,000 candidates → 200 seeds per run (configured), 184–189 after the multi-part filter
Context budget	8 passages for every clean and failure case
Generator, validation judge	Qwen3-32B (qwen.qwen3-32b-v1:0)
Evaluated models	nova-pro (amazon.nova-pro-v1:0); llama, Llama 3.3 70B (us.meta.llama3-3-70b-instruct-v1:0); gpt-oss, gpt-oss-120b (openai.gpt-oss-120b-1:0); all on Amazon Bedrock
Correctness judge	DeepSeek-V3.2, not an evaluated model and from a different family than the generator
Decoding	Evaluated models: temperature 0, ≤1,024 output tokens. Generation: temperature 0.2, ≤512. All judges and the baseline: temperature 0
Runs	Seeds 42, 123, 2026 on a frozen corpus; ≈3,190 failure cases per run, plus closed-book and conflict-aware probes; 3 models

is simply 1 if the model fails to abstain and 0 otherwise. An operator's drop is the mean over its cases (all three severities), and the Failure Robustness Score is

$$\mathrm{FRS} = 1 - \frac{1}{6}\sum_{o=1}^{6} \Delta_o, \tag{1}$$

where $\Delta_o \in [-1, 1]$ is operator o's mean drop as a proportion (tables report it in percentage points), so the six operators are weighted equally. FRS is a summary of one benchmark configuration (operator set, severity mix, context budget, conflict rule) and is meaningful for comparing runs under the same configuration, as the regression gate does; it is not an absolute measure of a model's robustness, and model rankings on it should be read together with the per-operator drops. We report mean ± std over the three runs and 95% cluster-bootstrap CIs pooled over runs (2,000 resamples of the 561 parent seeds; models share resamples, so model differences are paired [36]), and exact McNemar tests on identical items, Holm-corrected over 21 tests [37], [38].

Probes. A closed-book probe (no context) measures prior knowledge, and a conflict-aware prompt that permits flagging contradictions tests whether the default "shortest exact answer" instruction suppresses it. Both are excluded from FRS.

Scope. The reader receives fixed contexts; no retriever is run. Retrieval and reranking metrics such as Recall@k, MRR and nDCG are outside the current evaluation scope. Operators emulate the symptoms of retrieval, chunking and indexing defects at the reader.

V. Results

A. Construction funnel

Table IV reports the funnel. Acceptance is 74.8% ± 1.0%, up from 36.4% under an earlier title rule that rejected any question naming its subject and pushed the generator toward pronoun questions. Rules dominate rejections (211.7 ± 3.8 per run, 74.0 ± 3.6 unique); the uniqueness heuristic is the largest rule (90 per run). The judge and baseline each catch a few candidates nothing else catches (3.0 ± 2.0 and 2.7 ± 1.2 unique), and deduplication removes 2.3 per run. After the multi-part filter, the difficulty mix is 79/80/30, 77/77/30 and 78/78/32 (easy/medium/hard).

B. Human audit

A sample of 50 seeds (proportional to run × difficulty, balanced over categories) and 54 failure cases (three per operator × severity, one per run) was labeled with all automatic verdicts hidden (Table V). The primary annotator keeps 42 of 50 seeds (HAR 84.0%), against 66.0% in our earlier pilot. Of the eight not kept, two are

TABLE IV
Construction funnel, mean ± std over three runs; human rows are an audit sample (primary annotator) with 95% Wilson CIs. "Configured" rows are inputs.

Metric	Value
Candidates (configured)	1,000
Schema success	100.0% ± 0.0%
QA acceptance	74.8% ± 1.0%
Seeds selected (configured)	200
Seeds after multi-part filter	187.0 ± 2.6
Failure verification pass	94.9% ± 0.3%
Human acceptance (HAR), n=50	84.0% [71.5, 91.7]
Human failure validity, n=54	96.3% [87.5, 99.0]

TABLE V
Human audit by the primary annotator, with 95% Wilson CIs.

	Rate [95% CI]	Count
HAR (clean seeds)	84.0% [71.5, 91.7]	42/50
Valid injection (failure cases)	96.3% [87.5, 99.0]	52/54
conflict		7/9
other five operators		45/45
Answer-availability label correct	100% [93.4, 100]	54/54

present-tense facts that can go stale ("Where is Intuit Inc. headquartered?"), which the time rule misses because it matches words, not tense; four are unclear (an unstated context, two premises absent from the evidence, one ungrammatical question); one has no unique answer; and one only asks for an alternate name. Unresolved pronouns, 72% of earlier fixes, no longer occur. Failure cases are valid in 52 of 54; the two exceptions are conflict cases whose alternate is compatible with gold ("convection" as a heat-transfer mechanism). The answer-availability label is correct in all 54.

All audit figures are from this single primary annotator. Whether a case matches its severity level was hard to judge, mostly over how many context_noise distractors count as topically similar, so we treat severity as a configured property, not a human-validated one.

Judge validation. On a sheet of 270 model answers with the judge's verdicts hidden (15 per model × condition), the 255 whose seeds survive the multi-part filter agree with a human label in 91.0% of cases (Cohen's $\kappa = 0.72$ [39]); the judge accepts 17 answers the human rejects and rejects 6 the human accepts.

C. Model × operator performance

Clean accuracy is 0.98–0.99 for all models (Table VII); no pair differs on clean, missing_evidence, hard_negative, context_noise or evidence_position (McNemar, $p_{\text{Holm}} = 1$ for each pair except one at 0.85). FRS is 0.932, 0.939 and 0.924; under the reported conflict rule only llama − gpt-oss has a CI excluding zero (+0.015 [0.006, 0.022]). The FRS ranking of the models is not robust: it reverses under the stricter conflict rules (Table VI), because llama often names both values. We therefore do not rank the models by FRS. Two results hold under every rule: conflict is the largest drop for every model, and gpt-oss alone loses several points on chunk_boundary.

TABLE VI
Conflict paired drop (points) / FRS under three conflict scoring rules, pooled over three runs. The FRS ranking changes with the rule.

Conflict rule	nova-pro	llama	gpt-oss
Handled (reported)	28.5 / .932	26.6 / .939	30.4 / .924
Answer correctness	29.1 / .932	36.5 / .923	34.0 / .918
Gold only	38.9 / .915	61.7 / .880	47.8 / .895

TABLE VII
Score (mean ± std over runs) and paired drop in points with pooled 95% cluster-bootstrap CI. Score is accuracy, correct-abstention rate (abstain-expected) or handled rate (conflict).

Condition	nova-pro	llama	gpt-oss
clean	.98±.01	.99±.01	.98±.01
missing_evidence	.92±.02	.93±.02	.93±.01
	8.0 [6.6, 9.6]	6.8 [5.4, 8.3]	7.2 [5.8, 8.7]
hard_negative	.98±.01	.98±.01	.97±.01
	2.3 [1.3, 3.4]	2.1 [1.3, 3.1]	2.6 [1.5, 3.8]
conflict	.70±.03	.72±.02	.68±.02
	28.5 [25.6, 31.4]	26.6 [24.1, 29.3]	30.4 [27.6, 33.3]
context_noise	.98±.01	.99±.00	.99±.01
	0.1 [−0.9, 1.0]	−0.1 [−0.9, 0.7]	−0.3 [−1.0, 0.4]
chunk_boundary	.96±.01	.97±.01	.93±.00
	1.8 [0.7, 2.8]	1.2 [0.2, 2.4]	5.4 [3.8, 7.0]
evidence_position	.98±.00	.99±.00	.98±.01
	−0.1 [−1.1, 0.8]	−0.1 [−0.7, 0.4]	0.1 [−0.6, 0.8]
closed-book (probe)	.28±.03	.47±.02	.49±.03
FRS	.932 [.925, .940]	.939 [.933, .945]	.924 [.917, .931]

Five findings follow (Fig. 2, Tables VII–VIII). First, the absence operators work: models abstain on 92–93% of missing_evidence and 97–98% of hard_negative cases, with 6–7% and 2% hallucinated. Failing to abstain rises with how close the remaining context is to the removed evidence: 0.0, 7.8 and 15.7 points at low, medium and high severity. Second, conflict is the dominant failure for every model (26.6–30.4 points, rising from 12.2 to 28.9 to 44.4 with severity); the three CIs overlap, so it exposes a shared weakness rather than a ranking, and in 18–26% of cases the model silently adopts the planted value. Third, context_noise and evidence_position cause no drop with eight passages (every CI includes zero), while chunk_boundary grows with severity (0.4, 3.0, 5.8) and separates the models: gpt-oss loses 5.4 points, 3.8–4.6 more than the others ($p_{\text{Holm}} \leq 4 \times 10^{-7}$). On conflict, gpt-oss differs significantly only from llama (−4.1 points, $p_{\text{Holm}} = 0.015$). Fourth, how a model fails conflict is model-specific (Table VIII): llama names both values in 35% of cases, which a single-answer judge marks wrong, so llama is last on judge-only accuracy but first on the handled rate. Fifth, behavior depends on the prompt and prior knowledge: allowing the model to flag a conflict raises acknowledgement from 11–35% to 59–85% and the

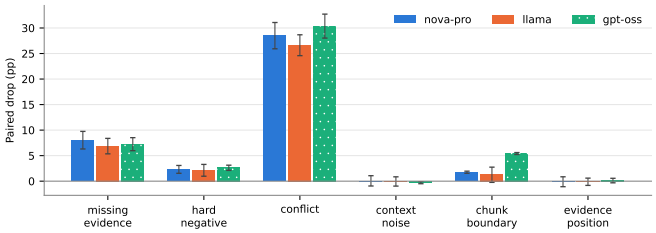


Fig. 2. Paired drop by operator, mean over three runs (± std). For abstain-expected operators the drop is the rate of failing to abstain. Every case has eight passages.

TABLE VIII
Conflict outcomes (%), default prompt unless noted, mean over three runs. Handled = gold, both, or acknowledged.

	nova-pro	llama	gpt-oss
Gold	59.3	36.8	50.4
Alternate	25.8	18.2	25.2
Both values	10.4	34.9	17.5
Acknowledged	10.6	35.1	17.6
Handled (judge-only)	69.7 (69.2)	72.0 (62.1)	67.9 (64.2)
Handled, prior known	86.5	78.0	78.2
Handled, prior unknown	63.2	66.6	58.1
Aware prompt: acknowledged	58.5	83.9	84.9
Aware prompt: handled	88.6	95.5	95.6

handled rate to 0.89–0.96, and seeds a model answers closed-book are handled 11–23 points more often than seeds it cannot.

D. Regression loop on two snapshots

Snapshot B re-fetches the same pages on 2026-10-28. In three months 580 of 768 pages have a new revision and 4 no longer resolve; 11,296 of A's 12,366 chunks are unchanged and B has 13,599. Of A's 600 seeds, 4.0–6.5% no longer hold on B (an edited supporting sentence or a changed answer), about 1.3–2.2% per month. Rebuilding the benchmark from B with the unchanged configuration takes 18 minutes and about 4.9k LLM calls; re-fetching the pages takes 188 s at 8 requests/s and chunking 16 s. The 3-model evaluation (3.3 h at a concurrency of 10) dominates the cost, which scales with models × cases rather than corpus size. B yields 769 accepted candidates and 188 seeds after the multi-part filter; only 18 of its 200 selected questions coincide with A's.

The gate compares a candidate run with a baseline per model × condition and reports FAIL when the score drops by more than 5 points with a 95% bootstrap CI below zero (paired when the benchmark is shared, unpaired when regenerated). Knowledge update (A→B, same system): all 21 cells pass (largest drop 1.5 points, all CIs include zero; FRS 0.935/0.945/0.924 on A, 0.938/0.955/0.928 on B). Decision: ship. Degraded retrieval depth (B, top-k 8→4, paired): clean accuracy is unchanged (+0.0 to +1.6 points), so a clean-only test would ship the change. The operators block it in 12 of 21 cells: evidence_position drops 65–66 points, context_noise 46–47, conflict 8–14 and chunk_boundary 8–10, all with CIs below zero, because these operators place the gold passage across all eight slots. FRS falls to 0.71–0.72; abstain-expected operators are unaffected. Decision: block. The two operators that show no effect on a correctly configured reader are the ones that catch this configuration error.

TABLE IX
What each operator measures, and what a regression may motivate inspecting in a production pipeline (not measured here).

Operator	Measured here	May motivate inspecting
missing_evidence	Abstention when evidence is absent (92–93%; 16-point failure at high severity)	Retrieval coverage, indexing
hard_negative	Abstention under near-miss passages (97–98%)	Retrieval precision, reranking
conflict	Handling of a contradicting copy (27–30-point drop)	Document lifecycle; answer-format prompt
chunk_boundary	Recombining a split sentence (1.2–5.4)	Chunk size, overlap
context_noise, evidence_position	No drop at 8 passages; 46–66 points at top-4	Retrieval depth, context assembly

VI. Discussion

A. What a regression points to

Table IX maps each operator to what it measures and what a regression would motivate a team to inspect. Because the benchmark presents fixed contexts, it measures the reader's behavior under each symptom, not the component that would produce the symptom in production. missing_evidence, for example, tests whether the generator abstains when supporting evidence is unavailable; in a production pipeline a similar symptom may motivate inspecting retrieval coverage, but the benchmark does not measure retriever recall.

B. Conflict

Gold evidence remains present, in full, next to a passage identical except for the answer, and all three models still lose 27–30 points, adopting the planted value in a fifth to a quarter of cases. The aware-prompt probe shows the models can detect the contradiction, but the usual "shortest exact answer" instruction suppresses it; the closed-book split shows part of the default-prompt accuracy is parametric recall. A stale but not-yet-purged document is therefore not something the reader can be relied on to adjudicate.

C. Limitations

The reader receives fixed contexts; no retriever or reranker is evaluated. The corpus is English, single-hop Wikipedia; the source adapter is designed for other stores (e.g. wikis or document repositories), but no enterprise connector is evaluated, and scaling claims rest on one 768-page corpus and the throughput measured in Section V-D.

The human audit covers 50 seeds and 54 failure cases from one primary annotator, so inter-annotator reliability remains unmeasured. A second annotator labeled the same rows but had seen the primary annotator's labels, so their agreement (49 of 50 decisions, $\kappa = 0.92$ [39]) does not answer this; an independent second annotation is future work. The conflict score and FRS depend on the scoring rule (Table VI). Severity levels are configured, not human-validated; 11% of medium missing_evidence cases keep no same-page passage. Context noise and evidence position may stress weaker models or longer contexts; we did not test them. The multi-part rule was applied to finished runs rather than at generation time, which leaves 184–189 seeds per run. Clean accuracy is near ceiling by design, which makes drops attributable to operators but leaves little room to separate difficulty tiers.

VII. Conclusion

RAGFailBench regenerates a failure-aware RAG benchmark whenever the corpus changes: validated, provenance-preserving seeds, a sampled human audit, six typed operators under a fixed context budget, and a per-operator regression gate. On a Wikipedia pilot it yields a stable funnel (74.8% acceptance), a human acceptance rate of 84% on a 50-seed sample, and a finding that holds under every scoring rule we tried: conflicting evidence is the largest failure for all three models. On two real snapshots it ships a corpus update and blocks a retrieval-depth change that clean accuracy cannot see. Code, configurations, all runs and the audit sheets are at https://github.com/taixingbi/RAGFailBench.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2024.

[3] P. Zhao, H. Zhang, Q. Yu, Z. Wang, Y. Geng, F. Fu, L. Yang, W. Zhang, and B. Cui, "Retrieval-augmented generation for AI-generated content: A survey," arXiv:2402.19473, 2024.

[4] Y. Huang and J. Huang, "A survey on retrieval-augmented text generation for large language models," arXiv:2404.10981, 2024.

[5] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-context retrieval-augmented language models," Trans. Assoc. Comput. Linguist., 2023.

[6] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in Proc. EACL (System Demonstrations), 2024.

[7] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An automated evaluation framework for retrieval-augmented generation systems," in Proc. NAACL, 2024.

[8] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in Proc. AAAI, 2024.

[9] R. Friel, M. Belyi, and A. Sanyal, "RAGBench: Explainable benchmark for retrieval-augmented generation systems," arXiv:2407.11005, 2024.

[10] Y. Lyu, Z. Li, S. Niu, F. Xiong, B. Tang, et al., "CRUD-RAG: A comprehensive Chinese benchmark for retrieval-augmented generation of large language models," arXiv:2401.17043, 2024.

[11] S. Shashidhar, C. Fourrier, A. Lozovskaya, T. Wolf, G. Tur, and D. Hakkani-Tür, "YourBench: Easy custom evaluation sets for everyone," arXiv:2504.01833, 2025.

[12] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," Trans. Assoc. Comput. Linguist., vol. 12, 2024.

[13] F. Cuconasu, G. Trappolini, F. Siciliano, S. Filice, C. Campagnano, Y. Maarek, N. Tonellotto, and F. Silvestri, "The power of noise: Redefining retrieval for RAG systems," in Proc. SIGIR, 2024.

[14] S. Longpre, K. Perisetla, A. Chen, N. Ramesh, C. DuBois, and S. Singh, "Entity-based knowledge conflicts in question answering," in Proc. EMNLP, 2021.

[15] K. Wu, E. Wu, and J. Zou, "ClashEval: Quantifying the tug-of-war between an LLM's internal prior and external evidence," arXiv:2404.10198, 2024.

[16] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in Proc. EMNLP, 2016.

[17] T. Kwiatkowski et al., "Natural Questions: A benchmark for question answering research," Trans. Assoc. Comput. Linguist., vol. 7, pp. 453–466, 2019.

[18] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," in Proc. ACL, 2017.

[19] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in Proc. EMNLP, 2018.

[20] F. Petroni et al., "KILT: A benchmark for knowledge intensive language tasks," in Proc. NAACL, 2021.

[21] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333–389, 2009.

[22] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense passage retrieval for open-domain question answering," in Proc. EMNLP, 2020.

[23] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," Trans. Mach. Learn. Res., 2022.

[24] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in Proc. SIGIR, 2020.

[25] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "ColBERTv2: Effective and efficient retrieval via lightweight late interaction," in Proc. NAACL, 2022.

[26] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in Proc. NeurIPS Datasets and Benchmarks Track, 2021.

[27] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG evaluation using GPT-4 with better human alignment," in Proc. EMNLP, 2023.

[28] LangChain, Inc., "LangSmith: Evaluation and observability platform for LLM applications," docs.smith.langchain.com, 2024.

[29] OpenAI, "Evals: A framework for evaluating LLMs and LLM systems," github.com/openai/evals, 2023.

[30] Confident AI, "DeepEval: The open-source LLM evaluation framework," github.com/confident-ai/deepeval, 2024.

[31] Promptfoo, "Open-source LLM testing and evaluation tool," github.com/promptfoo/promptfoo, 2024.

[32] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, et al., "Holistic evaluation of language models," arXiv:2211.09110, 2022.

[33] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive text embedding benchmark," in Proc. EACL, 2023.

[34] M. Fowler, "Continuous integration," martinfowler.com, 2006.

[35] G. Rothermel and M. J. Harrold, "Analyzing regression test selection techniques," IEEE Trans. Softw. Eng., vol. 22, no. 8, pp. 529–551, 1996.

[36] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. Chapman & Hall, 1993.

[37] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.
[38] S. Holm, "A simple sequentially rejective multiple test procedure," Scand. J. Statist., vol. 6, no. 2, pp. 65–70, 1979.
[39] J. Cohen, "A coefficient of agreement for nominal scales," Educ. Psychol. Meas., vol. 20, no. 1, pp. 37–46, 1960.
[40] Wikimedia Foundation, "Wikipedia: The free encyclopedia," https://www.wikipedia.org.
[41] Wikimedia Foundation, "MediaWiki API documentation," https://www.mediawiki.org/wiki/API:Main_page.